The Claude Efficiency System

> *A plain-English guide to better results from the same usage. The Leverage Point Co · Guide v1*

Last verified: 2026-10-16
Refresh when: new model, limit change, feature rename
Scope: the Claude apps (web, desktop, mobile) and Claude Code, on an individual Pro plan. Numbers in square brackets, like [3], point to the source list in Appendix A. A line marked **Practice** is my working method, not Anthropic guidance. The last section lists four things worth checking on your own account.

Who this is for, and the promise

For: operators and professionals who use Claude for real work and either hit usage limits or get average output.
Promise: a repeatable system that wastes less of your usage and gets better results from each message. It treats your plan the way an industrial engineer treats any capacity constraint: find the constraint, stop wasting it, plan work around it.

Terms you will need

- **Token:** a small chunk of text. Usage is measured in tokens, not messages.
- **API:** the developer interface to Claude. It is billed per token, separately from a subscription. [6]

- **Agentic:** Claude carrying out a multi-step job itself, using tools such as search or file editing, instead of answering one question. [6]
- **Thinking:** Claude working through a problem before it answers. [4]
- **Context window:** Claude's working memory for one chat. On paid plans, Anthropic says the newest models support up to 1 million tokens in chat. [1]
- **Effort:** a setting for how much work Claude puts into a response, including how much it thinks. Higher effort means more thorough answers, slower replies and more tokens. [4][8]
- **Connector:** a link from Claude to an outside app or data source, such as your files or calendar. [17]
- **Cache:** content Claude has already processed, which counts for less when you reuse it. [3]

---

1. The problem in one picture

Your plan is a capacity constraint with two buckets. Paid plans have a session limit that resets every five hours and a weekly limit that resets once a week. [1] Settings > Usage shows when your session and weekly limits reset. [3]

What you spend per task depends on message length, attachment size, conversation length, tool use (such as research and web search), model choice and artifact creation. [3] Anthropic also lists the features you use and the effort level you select. [1] The same pool is shared across the Claude apps and Claude Code. [1][20]

So efficiency has four levers, and the rest of this guide follows them in order:

1. Spend fewer tokens per task (one task per chat, clean context).
2. Right-size the model and effort to the task.
3. Avoid rework with better prompts.
4. Schedule work against the windows.

---

2. One chat = one task

Conversation length is one of the stated drivers of usage. [3] When a chat gets long enough to approach the context limit, Claude summarises earlier messages to keep going (this needs code execution turned on), and Anthropic notes that long chats which trigger this use more of your limit. Its advice is to start a new conversation when you are near a limit. [1]

The rule: a new task gets a new chat. A follow-up or a sub-step of the same task stays in the same chat.

Practice: the handoff note. Before you leave a chat that you will need to continue, ask Claude for a four-line summary: goal, decisions made, open questions, next step. Paste it into the new chat. You carry the conclusions without carrying the whole history.

Branching. To explore a different direction without losing your first attempt, edit an earlier message. This creates a separate version of the chat with its own artifacts. [16]

3. Context hygiene

Everything Claude has to look at counts: your instructions, attached files, tool descriptions and the chat so far. Anthropic lists these levers for keeping the window lean. [1]

- Keep project instructions short: general context, key guidelines and Claude's role. Put task-specific instructions in the chat itself.
- Remove project files you no longer use.
- Turn extended thinking off when the task does not need it, and lower the effort level for routine work.
- Turn off connected apps you do not need, and tell Claude not to search the web when you do not need current information.

Four habits that follow from the same pages:

- **Batch related questions into one message** rather than sending them one by one. [3]
- **Send a whole text for editing in one message,** not in pieces. [3]
- **Refer back to earlier information and files in the chat** instead of repeating or re-uploading them. [3]
- **Practice: paste only the relevant part.** If you need Claude to look at a log, a long email thread or a data export, trim it to the section that matters first. Anything pasted stays in context for the rest of the chat. Anthropic gives this exact advice for Claude Code [19]; it applies to chat for the same reason.

4. Choose the model and the effort by task

What the models are

Anthropic's current lineup has four models, each described with a different job. [6]

- **Haiku 5.5:** the fastest, for high-volume work where speed matters, such as classification, extraction and routing.
- **Sonnet 5.5:** the best mix of speed and intelligence.
- **Opus 5.5:** built for long-running agentic coding and knowledge work.
- **Fable 5.1:** for demanding reasoning and long-horizon agentic work. It is slower. [6]

All four have a 1M-token context window in the API. [6]

Cost. Anthropic's developer price list shows Opus 5.5 at twice the per-token price of Sonnet 5.5, with Haiku well below Sonnet. [6] That is the developer API, not your subscription. For subscriptions, Anthropic says model choice and effort level affect your usage [1], and its usage-credit tips say Haiku and the latest Sonnet cost fewer tokens than Opus models. [21] In Claude Code, Anthropic says Opus uses meaningfully more quota than Sonnet. [19] **Practice:** note the Current session bar on your Usage page before and after a heavy task. [3] You will see what that task cost you.

Fable on Pro. On Pro plans, Fable 5 and Fable 5.1 are not included in the plan's usage limits. They run on pay-as-you-go usage credits, a prepaid balance explained in Section 8. [22] Treat Fable as a deliberate purchase, not a default.

What effort and thinking do

- In the app, the model menu next to the send button holds the model, effort and thinking controls. [4]
- Effort covers every token in the response: text, tool calls and thinking. [8] Anthropic's developer documentation lists five levels: low, medium, high, xhigh and max. [8]
- In the app, Anthropic says low and medium suit routine tasks and stretch your usage, high is the best balance, and max is for difficult, correctness-critical work. [4]
- Thinking is a separate switch from effort in the app. [4] On Opus 5.5 and Fable 5.1, thinking is always on and adaptive, meaning Claude decides when and how much to think. On Sonnet 5.5 and Haiku 5.5 it is adaptive. [6]
- In the API, the default effort is high on most models and medium on Opus 5.5 and Haiku 5.5. [8] Your app menu marks each model's default. [4]
- Changes apply from Claude's next response, so you can adjust mid-chat. [4]

The decision table

Task	Start with	Effort	Why
Quick lookups, reformatting, sorting, extraction, simple edits	Haiku 5.5	Low to medium	Haiku is positioned for high-volume work where speed matters. [6] At low effort in long, multi-step prompts it is more likely to skip a search or stop early. [8]
Everyday drafting, analysis, spreadsheets, research summaries	Sonnet 5.5	Medium, or low for quick chat-style work. Raise to high for harder or longer tasks.	Sonnet is the speed-and-intelligence default. [6] For chat, Anthropic suggests starting at medium or low. For multi-step agentic work it suggests medium for well-defined tasks and high for harder or longer ones. [8][10]
Hard, long, multi-step work: large document sets, complex analysis, long-running tasks	Opus 5.5	Medium (its API default). Raise only if quality needs it.	Positioned for long-running knowledge work. [6] Anthropic reports medium on Opus 5.5 matching high on Opus 5 in its own tests. [11]
Anything your Opus run still fails	Fable 5.1	High	On Pro this costs usage credits. [22]

Move up one step only after a clear failure, and move effort before you move model. Anthropic makes the same point: tuning effort is often a better lever than switching models. [7] If you want less thinking, lower the effort; this works more reliably than asking Claude in the prompt to think less. [11]

What lower effort costs you. On long agentic tasks, Sonnet 5.5 at low or medium effort is more likely to stop and check in before it finishes. [10] That is fine for short work and a reason to raise effort for long jobs.

5. Prompts that work

Anthropic's own advice has one starting point: define what a good result looks like before you tune the prompt. [12] Then use these techniques, all from its published guidance. [9]

- **Be specific.** Treat Claude like a capable new hire who does not know your norms. Test: would a colleague with no context be confused by your prompt? [9]
- **Give the reason.** A constraint with a reason ("this will be read aloud, so no ellipses") generalises better than a bare rule. [9]
- **Show examples.** Three to five relevant, varied examples, wrapped in simple tags, improve accuracy and consistency. [9]
- **Separate the parts.** Simple tags such as `<document>` and `<instructions>` stop Claude confusing your material with your request. [9]
- **Put long material first, the question last.** In Anthropic's tests, putting the query at the end improved quality by up to 30 percent on complex inputs. For long documents, ask Claude to pull the relevant quotes first. [9]
- **Say what you want, not what to avoid.** "Write in flowing paragraphs" beats "no bullet points". [9]
- **Ask for the action.** "Can you suggest changes?" tends to get suggestions. "Make these changes" gets changes. [9]
- **Add a check.** "Before you finish, verify the answer against these criteria" catches errors. [9]

Two more that matter for business work, specific to Sonnet 5.5:

- It may answer from what it already knows when a search would catch details that have changed, such as what is allowed, required or charged. Ask it to check current specifics. [10] Searching uses usage [3], so ask only when currency matters.
- On open-ended requests it may start building when you wanted ideas. Say "give me options and stop" if that is what you want. [10]

The five-part prompt. Task · Context · Input · Output · Check. The three examples below are generic small-business workflows with no client data.

Example A: monthly expense sort

Before

> Here's my bank export. Sort out my expenses for the month.

What goes wrong: no categories, no format, no check. You get a plausible table and spend three more messages fixing it.

After

```
Task: categorise this month's business expenses so I can reconcile
them against my budget sheet.
Context: I run a small digital-products business. Categories:
software, contractors, ads, bank fees. Anything that looks
personal: flag it, don't categorise it.
Input: the attached CSV.
Output: a table with date, description, amount, category,
confidence (high / medium / low). Then a short list of lines you
could not categorise, and why. Then totals per category.
Check: before you finish, confirm the category totals add up to the
file total, and tell me if they don't.
```

Why it works: success criteria are stated [12], context and format are explicit [9], and a self-check is built in [9]. Haiku or Sonnet at low to medium effort is enough for this one.

Example B: reviewing a long document

Before (the document pasted, then the question underneath)

> Is this proposal any good?

After

```
<document>
[paste the proposal]
</document>

I'm sending this to a first-time client with a fixed budget.
First, pull the three sentences that carry the offer and the price.
Then judge the proposal on: clarity of scope, risk to the client,
and whether the price is justified on the page.
For each, give a verdict (strong / adequate / weak), the evidence,
and one specific rewrite.
Stop there. Don't rewrite the whole document.
```

Why it works: the document comes first and the question last [9], quotes come first [9], the audience is the reason behind the criteria [9], and the last line limits scope [10].

Example C: planning before building

Before

> Help me with my product launch.

After

```
I'm launching a small digital product in three weeks. I work on it
about ten hours a week.
Give me three options for sequencing the work, with the trade-off
of each, and recommend one.
Check current platform fees and rules with search; don't answer
those from what you already know.
Then stop. Don't draft any copy until I pick an option.
```

Why it works: it asks for options, not output [10], it forces a search for details that change [10], and it states where Claude stops [10].

---

6. Files, projects, memory

Files

- **Types:** PDF, DOCX, CSV, TXT, HTML, ODT, RTF, EPUB, JSON and XLSX (XLSX needs an account setting turned on), plus JPEG, PNG, GIF and WebP images. [15]
- **Chat limits:** 30 MB per file and up to 20 files per chat. [15]
- **PDFs:** under 100 pages, Claude reads both text and visuals. Non-PDF documents are read as text only, so images embedded in a Word file are not seen. [15]
- **Page numbers:** refer to the page numbers your PDF viewer shows, not the ones printed on the page. [15]

Projects

A project is a workspace with its own chat history and knowledge base, plus instructions you set once. [13] On paid plans, when project knowledge nears the context limit Claude switches to retrieval (it loads only the relevant parts), which expands capacity by up to ten times. [13]

Project content is cached, so reusing it counts for less against your limits. [3]

Memory and past chats

You can ask Claude to search your earlier conversations. It shows up as a tool call, and incognito chats are not searched. Claude can also keep a memory generated from your chats, centred on work context such as your role and projects. You can update it from a chat. [14] Each project has its own separate memory space, kept apart from your other projects and from chats outside projects. [14] You can turn both features off in Settings > Capabilities. Pausing memory keeps what Claude already holds but stops it using or adding to it. Resetting memory deletes it permanently. [14] Paid plans can use past-chat search to avoid re-explaining things. [3]

Which one to use

What it is	Where it belongs
A stable fact about you or how you work	Memory [14]
Materials and standing instructions for a body of work	A project [13]
This task, right now	A chat (Section 2)
A one-off reference	An uploaded file in the chat [15]

Artifacts. Claude creates an artifact when content is substantial, self-contained and something you will edit or reuse. Anthropic gives "over 15 lines" as a typical size. [16] Artifact creation and use count toward your usage. [3] On paid plans you can also start from Docs, Slides or Design templates, and Docs export to Word, PDF, Markdown and Google Docs.

[16] **Practice:** if you will read an answer once and move on, ask for it in the chat, not as an artifact.

---

7. Connectors and tools

Connectors let Claude read from, and act in, your outside tools. Custom connectors are in beta on Pro, Max, Team and Enterprise. [17]

- **They cost context.** Anthropic calls tools and connectors token-intensive. Switch off the ones a conversation does not need. [1] You can enable connectors per conversation. [17]
- **Be careful what you connect.** Connect only servers you trust, review the permissions requested, expect that a malicious one may hide instructions meant to steer Claude, and watch for unexpected changes in tool behaviour. [17]
- **Research mode is a heavy user.** It can call your connectors without approval at each step, and Anthropic advises disabling any connector tool that can write to an outside app while you use it. [17] Research also consumes tokens faster than normal chat. [21]
- **Web search counts as tool use** in your usage. [3]

Practice: connectors off by default, switched on for the task that needs them.

---

8. Limits, and how to plan around them

What Anthropic says

- Two limits: a five-hour session limit and a weekly limit. [1]

- Settings > Usage shows when your session limit and your weekly limit reset. [3]
- This guide does not confirm when a window starts or how the weekly reset is counted, so read your reset time on that page instead of calculating it.
- When you reach a limit you can wait, upgrade to Max, or turn on usage credits. [1][2][20]
- Usage credits are pay-as-you-go at standard API rates, billed separately from your subscription. You can set a monthly spend cap, and your regular limits keep resetting on schedule. [21]
- Anthropic's advice on credits: use your plan's included usage first, and plan intensive sessions around your five-hour windows. [21]
- Pro includes Claude Code access, and the limits are shared. [20]

How to plan (Practice, not Anthropic guidance)

An industrial engineer would treat this as a scheduling problem with two buckets and a hard reset.

1. **Know your reset time.** Check the Usage page once and note your weekly reset. [3]
2. **Order work small to large.** Quick, cheap tasks first; the heavy reasoning job when you have the most headroom.
3. **Run the biggest job right after a reset,** not at the end of a window or a week.
4. **Never stretch one task across a reset.** A task cut in half by a limit stalls mid-work and is slower to restart.
5. **Keep a reserve.** Do not spend the last 10 percent of a window on anything you cannot afford to leave half-done.

6. **Batch.** One message with five related questions beats five messages. [3]
7. **Escalate by exception.** Default to Sonnet at medium effort. Raise effort first, then model, only on failure. [7]

---

9. When to move to Claude Code

Claude Code is Anthropic's tool for work that touches files and commands. It reads a codebase, edits files, runs commands and connects to development tools. It runs in a terminal, in code editors, in the desktop app and in a browser. [18] It is included with Pro. [20]

Move to Claude Code when:

- The work lives in a folder or code repository (a project folder with version history) and needs files read, edited or created in place. [18]
- The task is a script or a multi-step job you will run more than once. [18]
- You want a long task to run while you are away. Claude Code on the web starts long-running tasks and lets you check back later. [18]
- You want something to run on a schedule. [18]

Stay in chat when: the job is one-shot thinking, writing or analysis and needs no files on your machine.

The same efficiency rules apply, with sharper edges. Usage is shared with chat. [20] Every turn re-sends the whole conversation, plus your standing instructions file (called `CLAUDE.md`) and any files Claude has read. [19]

- `/clear` starts a fresh conversation. Use it between tasks. `/compact` summarises the conversation to keep going mid-task. `/context` shows what is loaded. `/model` shows and switches models. [19]
- Sonnet for most work, Opus for harder problems such as large code rewrites or difficult debugging, Haiku for quick mechanical jobs. `/model opusplan` uses Opus to plan and Sonnet to carry it out. [19]
- Point Claude at a file by path instead of pasting it. Trim logs to the relevant 20 to 30 lines. [19]
- Keep `CLAUDE.md` under roughly 200 lines, and add a note only the second time you correct the same thing. [19]
- For anything that touches more than two or three files, ask for a plan first. [19]
- **Watch for this:** if a setting named `ANTHROPIC_API_KEY` is stored on your computer (an environment variable), Claude Code uses it instead of your subscription, and you pay API charges. [20]

---

10. Cheat sheet

The eight rules

1. One chat = one task. New task, new chat. [1][3]
2. Hand off with a four-line summary, not the whole history. (Practice)
3. Batch related questions. Refer back to the chat instead of re-sending files. [3]
4. Switch off connectors, web search and thinking you do not need. [1]
5. Default to Sonnet 5.5 at medium effort. Raise effort first, then model, only on failure. (Practice, built on [6][7][10])

6. Prompt in five parts: Task · Context · Input · Output · Check. Long material first, question last. [9]
7. Use a project for materials you reuse; memory for stable facts about you. [13][14]
8. Run the biggest job right after a reset. Never stretch a task across one. (Practice)

Model and effort

Job	Model	Effort
Sort, extract, reformat	Haiku 5.5	Low to medium
Draft, analyse, summarise	Sonnet 5.5	Medium
Hard, long, multi-step	Opus 5.5	Medium, then up
Still failing	Fable 5.1 (usage credits on Pro [22])	High

Limits

- Two buckets: five-hour session and weekly. [1]
- Shared by chat and Claude Code. [1][20]
- What raises spend: length, attachments, long chats, tools, higher effort, bigger model, artifacts. [1][3]
- At a limit: wait, upgrade, or usage credits with a spend cap. [1][2][21]

Claude Code in four lines

- Files, commands, repeat jobs, scheduled jobs → Claude Code. [18]
- `/clear` between tasks, `/compact` mid-task. [19]
- Plan with Opus, build with Sonnet: `/model opusplan`. [19]
- No `ANTHROPIC_API_KEY` set, unless you mean to pay API rates. [20]

Check this yourself

Four details depend on your own account. Each takes a minute.

1. **Your model menu.** Open the menu next to the send button and see which models offer an effort setting and which levels they list. Anthropic's help page on these settings is written around earlier models [4], so your menu is the reference for your account.
2. **Your weekly limits.** Open Settings > Usage and note which weekly limits your plan shows and when each resets. Anthropic's pages describe the weekly limit in more than one way [2][3].
3. **Your session window.** Send your first message of the day, then check Current session on the Usage page to see how much of the five hours remains. [3]
4. **Your context window.** Anthropic says the newest models support up to 1M tokens in chat on paid plans [1], and publishes a per-model breakdown [5]. Check it for the model you use before you plan around a very long chat.

Appendix A: Sources

All pages were read on 2026-10-11.

#	Source
1	How do usage and length limits work? support.claude.com/en/articles/11647753
2	What is the Pro plan? support.claude.com/en/articles/8325606
3	Usage limit best practices. support.claude.com/en/articles/9797557
4	Change the model, effort, and thinking settings. support.claude.com/en/articles/8664678
5	How large is the context window on paid Claude plans? support.claude.com/en/articles/8606394
6	Models overview. platform.claude.com/docs/en/models/overview
7	Choosing the right model. platform.claude.com/docs/en/about-claude/models/choosing-a-model
8	Effort. platform.claude.com/docs/en/build-with-claude/effort
9	Prompting best practices. platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
10	Prompting Claude Sonnet 5.5. platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-sonnet-5-5
11	Prompting Claude Opus 5.5. platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-opus-5-5
12	Prompt engineering overview. platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
13	What are projects? support.claude.com/en/articles/9517075
14	Use Claude's chat search and memory to build on previous context. support.claude.com/en/articles/11817273
15	Uploading files to Claude. support.claude.com/en/articles/8241126
16	What are artifacts and how do I use them? support.claude.com/en/articles/9487310

#	Source
17	Getting started with custom connectors using remote MCP. support.claude.com/en/articles/11175166
18	Claude Code overview. code.claude.com/docs/en/overview
19	Models, usage, and limits in Claude Code. support.claude.com/en/articles/14552983
20	Use Claude Code with your Pro or Max plan. support.claude.com/en/articles/11145838
21	Manage usage credits for paid Claude plans. support.claude.com/en/articles/12429409
22	Claude Fable models on your plan. support.claude.com/en/articles/15424964

The Leverage Point Co · One tool at a time · More free tools: https://systemyxventures.com/tools/